

Controllo del flusso

Break e continue

Come interrompere o saltare un passaggio dentro un ciclo.

Due comandi per controllare un ciclo

Dentro un ciclo puoi avere bisogno di fermarti prima o saltare un giro. PHP offre due parole utili:

- `break` interrompe il ciclo
- `continue` salta al giro successivo

Uscire con break

Immagina di cercare un nome in una lista. Quando lo trovi, non serve continuare.

```
<?php
$nomi = ["Luca", "Sara", "Marta"];

foreach ($nomi as $nome) {
    if ($nome === "Sara") {
        echo "Trovata Sara";
        break;
    }
}
```

Appena PHP incontra `break`, esce dal ciclo.

Saltare un giro con continue

`continue` non chiude il ciclo. Salta solo il resto del blocco corrente.

```
<?php
$numeri = [1, 2, 3, 4, 5];

foreach ($numeri as $numero) {
    if ($numero % 2 === 0) {
        continue;
    }

    echo $numero . "\n";
}
```

Output:

```
1
3
5
```

I numeri pari vengono saltati.

Quando usarli

`break` è utile quando hai già ottenuto quello che cercavi. `continue` è utile quando un elemento non ti interessa, ma vuoi controllare gli altri.

Attenzione: se usi troppi `break` e `continue`, il ciclo può diventare difficile da leggere. Usali quando rendono il codice più chiaro.

break in un ciclo for

`break` non funziona solo con `foreach`. Puoi usarlo anche con `for` e `while`.

```
<?php
for ($numero = 1; $numero <= 10; $numero++) {
    if ($numero === 4) {
        break;
    }

    echo $numero . "\n";
}
```

Output:

```
1
2
3
```

Quando `$numero` diventa `4`, il ciclo si ferma.

continue per filtrare dati

`continue` è utile quando vuoi ignorare alcuni elementi prima di fare il lavoro principale.

```
<?php
$nomi = ["Luca", "", "Sara"];

foreach ($nomi as $nome) {
    if ($nome === "") {
        continue;
    }

    echo "Ciao $nome\n";
}
```

La stringa vuota viene saltata. Il resto del ciclo lavora solo sui nomi validi.

Domanda guida

Prima di usare uno dei due comandi chiedi: voglio fermare tutta la ripetizione o saltare solo questo elemento? La risposta ti dice se scegliere `break` o `continue` .

Condizioni

Come far prendere decisioni a un programma PHP con if, else e switch.

A cosa servono le condizioni

Una condizione è un bivio. Se succede una cosa, il programma fa un'azione. Altrimenti ne fa un'altra.

if

`if` esegue un blocco solo quando la condizione è vera.

```
<?php
$eta = 20;

if ($eta >= 18) {
    echo "Sei maggiorenne";
}
```

La condizione sta tra parentesi tonde. Le istruzioni da eseguire stanno tra parentesi graffe.

else

`else` copre il caso contrario.

```
<?php
$eta = 16;

if ($eta >= 18) {
    echo "Sei maggiorenne";
} else {
    echo "Sei minorenne";
}
```

elseif

`elseif` permette di controllare piu casi.

```
<?php
$voto = 7;

if ($voto >= 9) {
    echo "Ottimo";
} elseif ($voto >= 6) {
    echo "Sufficiente";
} else {
    echo "Da ripassare";
}
```

PHP controlla dall'alto verso il basso e si ferma al primo blocco valido.

switch

`switch` e comodo quando confronti lo stesso valore con piu possibilita.

```
<?php
$giorno = "sabato";

switch ($giorno) {
    case "sabato":
    case "domenica":
        echo "Weekend";
        break;
    default:
        echo "Giorno lavorativo";
}
```

break evita che PHP continui nei casi successivi. **default** gestisce tutto quello che non è stato previsto prima.

Condizioni booleane

Una condizione deve diventare **true** o **false**.

```
<?php
$utenteAttivo = true;

if ($utenteAttivo) {
    echo "Accesso consentito";
}
```

Quando il nome è chiaro, il codice si legge quasi come una frase.

Combinare più controlli

Una decisione può dipendere da più condizioni.

```
<?php
$eta = 20;
$haDocumento = true;

if ($eta >= 18 && $haDocumento) {
    echo "Ingresso consentito";
} else {
    echo "Ingresso non consentito";
}
```

`&&` richiede che entrambe le condizioni siano vere. Se una delle due è falsa, PHP passa a `else` .

Rendere leggibile una condizione

Quando una condizione diventa lunga, puoi salvarne una parte in una variabile con un nome chiaro.

```
<?php
$totale = 75;
$clienteRegistrato = true;

$haDirittoSconto = $totale > 50 && $clienteRegistrato;

if ($haDirittoSconto) {
    echo "Sconto applicato";
}
```

Il programma fa la stessa cosa, ma è più facile da leggere.

Errore comune: troppi livelli annidati

Se metti un `if` dentro un `if` dentro un altro `if` , il codice diventa faticoso. Spesso puoi usare `elseif` , variabili booleane o funzioni piccole per chiarire la logica.

Prova tu

Scrivi un controllo per un ordine: se il totale è almeno 30 euro e il prodotto è disponibile, mostra `Ordine confermato` . Altrimenti mostra `Ordine non disponibile` .

Cicli

Come ripetere istruzioni con while, do while, for e foreach.

Cosa sono i cicli

Un ciclo ripete una o più istruzioni. È utile quando non vuoi scrivere la stessa riga molte volte.

while

`while` ripete finché una condizione resta vera.

```
<?php
$numero = 1;

while ($numero <= 3) {
    echo $numero . "\n";
    $numero++;
}
```

Output:

```
1
2
3
```

Senza `$numero++`, la condizione resterebbe sempre vera e il ciclo non finirebbe.

do while

`do while` esegue il blocco almeno una volta, poi controlla la condizione.

```
<?php
$tentativi = 0;

do {
    echo "Provo...\n";
    $tentativi++;
} while ($tentativi < 1);
```

for

for è comodo quando sai quante ripetizioni vuoi fare.

```
<?php
for ($i = 1; $i <= 5; $i++) {
    echo "Riga $i\n";
}
```

Dentro le parentesi ci sono: valore iniziale, condizione, aggiornamento.

foreach

foreach serve per scorrere array.

```
<?php
$nomi = ["Luca", "Sara", "Marta"];

foreach ($nomi as $nome) {
    echo "Ciao $nome\n";
}
```

A ogni giro, **\$nome** contiene un elemento diverso dell'array.

Scegliere il ciclo giusto

Usa `foreach` per gli array. Usa `for` quando conosci il numero di ripetizioni. Usa `while` quando vuoi ripetere finché una condizione cambia.

Evitare cicli infiniti

Un ciclo infinito continua per sempre. Di solito succede quando la condizione non cambia mai.

```
<?php
$numero = 1;

while ($numero <= 3) {
    echo $numero;
}
```

Qui `$numero` resta sempre `1`, quindi la condizione resta vera. Per correggerlo devi aggiornare il valore dentro il ciclo.

```
<?php
$numero = 1;

while ($numero <= 3) {
    echo $numero . "\n";
    $numero++;
}
```

Costruire un totale con foreach

Un uso molto comune dei cicli è sommare valori.

```
<?php
$prezzi = [10, 15, 5];
$totale = 0;

foreach ($prezzi as $prezzo) {
    $totale += $prezzo;
}

echo $totale;
```

Output:

30

Prova tu

Crea un array di voti e usa `foreach` per sommarli. Poi dividi il totale per `count($voti)` e stampa la media.